# Fluxo de Caixa Diário

Documento de arquitetura de soluções

Autor: Daniel Pereira
Versão 1.0 · 5 de novembro de 2026
Status: arquitetura e código prontos; execução dos testes e medições pendentes

# Sumário

**1. Resumo executivo** ..... **3**
Como o documento atende ao desafio ..... 3
**2. Entendimento do problema** ..... **4**
Premissas a validar com o negócio ..... 4
Fora do escopo ..... 4
**3. Domínios e capacidades de negócio** ..... **5**
3.1 Cadeia de valor ..... 5
3.2 Mapa de capacidades ..... 5
3.3 Contextos delimitados ..... 5
3.4 Linguagem ubíqua ..... 5
3.5 Eventos de domínio ..... 6
**4. Requisitos refinados** ..... **7**
4.1 Requisitos funcionais ..... 7
4.2 Regras de negócio ..... 7
4.3 Requisitos não funcionais ..... 7
4.4 Cenários de falha ..... 8
**5. Arquitetura alvo** ..... **9**
5.1 Contexto do sistema ..... 9
5.2 Containers ..... 9
5.3 Fluxo de registro ..... 10
5.4 Fluxo de consolidação ..... 10
5.5 Consulta do saldo ..... 10
5.6 Modelo de dados ..... 11
5.7 APIs ..... 11
5.8 Estratégia de integração ..... 11
5.9 Implantação na AWS ..... 12
**6. Decisões arquiteturais** ..... **13**
ADR-0001: Dois serviços independentes ..... 13
ADR-0002: Consolidação assíncrona com Transactional Outbox ..... 13
ADR-0003: RabbitMQ como broker ..... 13
ADR-0004: Consumidor idempotente e somas comutativas ..... 14
ADR-0005: Modelo de leitura com cache Redis ..... 14
ADR-0006: Lançamentos imutáveis e idempotência na API ..... 14
ADR-0007: Java 21, Spring Boot e PostgreSQL ..... 15
ADR-0008: OAuth2/JWT e isolamento por comerciante ..... 15
**7. Atributos de qualidade** ..... **16**
7.1 Escalabilidade e desempenho ..... 16
7.2 Resiliência ..... 16
7.3 Segurança e critérios de consumo ..... 17
**8. Observabilidade** ..... **18**
**9. Estratégia de testes** ..... **19**
Provas dos requisitos do desafio ..... 19
**10. Arquitetura de transição** ..... **20**
Fases da migração ..... 21
Cuidados ..... 21
**11. Estimativa de custos** ..... **22**
Alavancas de redução ..... 22
**12. Repositório e execução local** ..... **23**
Estrutura ..... 23
Execução local ..... 23
**13. Riscos e evoluções futuras** ..... **24**
Riscos ..... 24
Evoluções ..... 24

# 1. Resumo executivo

Este documento descreve a arquitetura proposta para que um comerciante controle o fluxo de caixa diário, com registro de débitos e créditos e um relatório de saldo diário consolidado.

O ponto central da solução é que as duas capacidades têm criticidades diferentes. **Um lançamento não registrado é informação perdida; um relatório atrasado pode ser recuperado.** Por isso, os serviços são separados e se comunicam por eventos, de modo que o registro nunca dependa do relatório.

| Tema | Decisão |
|---|---|
| Estilo | Dois serviços independentes, cada um com o próprio banco, integrados por eventos |
| Isolamento de falhas | Transactional Outbox e RabbitMQ: Lançamentos segue operando com o Consolidado ou o broker fora do ar |
| Pico de 50 req/s | Modelo de leitura próprio, cache Redis, serviços sem estado e escala horizontal |
| Confiabilidade | Entrega pelo menos uma vez, consumidor idempotente e somas comutativas |
| Integridade | Lançamentos imutáveis, correção por estorno e idempotência na API |
| Tecnologia | Java 21, Spring Boot, PostgreSQL, RabbitMQ, Redis e, no alvo, AWS |
| Custo do alvo | Ordem de grandeza de US$ 1.450 por mês, sem custo de licenças |

## Como o documento atende ao desafio

| Requisito | Onde está | Situação |
|---|---|---|
| Domínios funcionais e capacidades de negócio | Seção 3 | Documentado |
| Requisitos funcionais e não funcionais refinados | Seção 4 | Documentado |
| Desenho da arquitetura alvo | Seção 5 | Documentado |
| Justificativa de ferramentas e tipo de arquitetura | Seção 6 | Documentado |
| Código na linguagem escolhida | Repositório | Implementado em Java; execução a validar |
| Testes | Seção 9 e repositório | Escritos; execução e carga a validar |
| README e execução local | Seção 12 e repositório | Escrito; a validar |
| Repositório público no GitHub | Seção 12 | A publicar |
| Diferencial: arquitetura de transição | Seção 10 | Documentado |
| Diferencial: custos de infraestrutura e licenças | Seção 11 | Estimado |
| Diferencial: monitoramento e observabilidade | Seção 8 | Documentado |
| Diferencial: segurança para consumo de serviços | Seção 7.3 | Documentado |

# 2. Entendimento do problema

O desafio pede dois serviços: controle de lançamentos e consolidado diário. Também estabelece dois requisitos não funcionais explícitos:

- O serviço de lançamentos não pode ficar indisponível se o consolidado cair.
- Em dias de pico, o consolidado recebe 50 requisições por segundo, com no máximo 5% de perda.

Todas as demais metas deste documento são propostas de projeto e estão identificadas como tal.

## Premissas a validar com o negócio

| # | Premissa | Se estiver errada |
|---|---|---|
| P1 | Os 50 req/s se referem a consultas ao consolidado | Rever o dimensionamento do consumidor; o teste de ingestão de eventos já cobre essa leitura |
| P2 | Máximo de 5% de perda significa pelo menos 95% de respostas bem-sucedidas no pico, ou 47,5 req/s | Ajustar metas e teste de carga |
| P3 | Um atraso de segundos no consolidado é aceitável | Rever a consolidação assíncrona (ADR-0002) |
| P4 | Lançamentos retroativos são permitidos; datas futuras, não | Ajustar a regra RN04 |
| P5 | Moeda única (BRL) e um fuso horário por comerciante | Incluir a moeda no modelo e na chave do consolidado |
| P6 | Saldo negativo é permitido, pois o sistema registra fatos e não autoriza pagamentos | Criar regra de validação |

## Fora do escopo

Múltiplas moedas, conciliação bancária, integração com adquirentes, contabilidade formal e cadastro de comerciantes.

# 3. Domínios e capacidades de negócio

## 3.1 Cadeia de valor

Movimentação financeira → registrar lançamento → consolidar por dia → consultar saldo → decisão do comerciante.

## 3.2 Mapa de capacidades

| Domínio | Tipo | Capacidade | Criticidade |
|---|---|---|---|
| Lançamentos | Core | Registrar débito e crédito | Alta |
| Lançamentos | Core | Estornar lançamento | Alta |
| Lançamentos | Core | Consultar lançamentos | Média |
| Lançamentos | Core | Publicar fatos de negócio | Alta |
| Consolidação | Core | Consolidar movimentos por dia | Média |
| Consolidação | Core | Consultar saldo diário | Média, com pico de 50 req/s |
| Consolidação | Core | Reprocessar o consolidado | Média |
| Identidade e acesso | Genérico | Autenticar e autorizar | Alta (comprar, não construir) |
| Observabilidade | Suporte | Métricas, logs, traces e alertas | Alta |

## 3.3 Contextos delimitados

- **Lançamentos** é a fonte da verdade e não conhece o Consolidado.
- **Consolidação** mantém uma visão derivada, que pode ser reconstruída a partir dos lançamentos.
- **Identidade** é um serviço genérico que fornece o comerciante autenticado aos demais contextos.

A relação entre Lançamentos e Consolidação é de fornecedor e cliente. O contrato entre eles é o evento publicado, nunca o banco de dados do outro serviço.

## 3.4 Linguagem ubíqua

| Termo | Definição |
|---|---|
| Comerciante | Dono do fluxo de caixa e fronteira de isolamento dos dados |
| Lançamento | Registro imutável de um crédito ou débito |
| Data de competência | Dia, no fuso do comerciante, ao qual o lançamento pertence |
| Estorno | Novo lançamento de sentido inverso que anula um lançamento anterior |
| Saldo do dia | Total de créditos menos total de débitos do dia |
| Saldo acumulado | Soma dos saldos do dia até a data consultada |

## 3.5 Eventos de domínio

| Evento | Consumidor | Efeito |
|---|---|---|
| `LancamentoRegistrado` | Consolidação | Soma crédito ou débito no dia de competência |
| `LancamentoEstornado` | Consolidação | Soma o movimento inverso no dia de competência do estorno |

Exemplo de contrato:

```
{
  "eventId": "0192f0a1-7c3e-7b52-9a10-3f1c2d4e5a6b",
  "eventType": "LancamentoRegistrado",
  "schemaVersion": 1,
  "occurredAt": "2026-11-03T14:30:00Z",
  "comercianteId": "c-123",
  "lancamentoId": "0192f0a1-7c3d-7000-8000-000000000001",
  "tipo": "CREDITO",
  "valorCentavos": 15000,
  "moeda": "BRL",
  "dataCompetencia": "2026-11-03"
}
```

# 4. Requisitos refinados

## 4.1 Requisitos funcionais

| ID | Requisito | Serviço |
|---|---|---|
| RF01 | Registrar lançamento de crédito ou débito | Lançamentos |
| RF02 | Estornar lançamento existente | Lançamentos |
| RF03 | Consultar um lançamento e listar por período, com paginação | Lançamentos |
| RF04 | Atualizar o consolidado a partir de eventos | Consolidado |
| RF05 | Consultar créditos, débitos, saldo do dia e saldo acumulado por data | Consolidado |
| RF06 | Consultar o consolidado por período | Consolidado |
| RF07 | Reconstruir o consolidado a partir dos lançamentos (evolução) | Consolidado |

## 4.2 Regras de negócio

| ID | Regra |
|---|---|
| RN01 | O valor é sempre positivo; o sentido vem do tipo (`CREDITO` ou `DEBITO`) |
| RN02 | Valores são armazenados em centavos inteiros, nunca em ponto flutuante |
| RN03 | Lançamentos são imutáveis: não há edição nem exclusão |
| RN04 | A data de competência não pode estar no futuro, considerando o fuso do comerciante |
| RN05 | Correções são feitas por estorno, com tipo inverso, vínculo ao lançamento original e data de competência igual à do estorno |
| RN06 | Um lançamento pode ser estornado uma única vez, e um estorno não pode ser estornado |
| RN07 | Repetir uma requisição com a mesma `Idempotency-Key` retorna o resultado original, sem novo lançamento |
| RN08 | Reutilizar a chave com conteúdo diferente retorna `409 Conflict` |
| RN09 | Um comerciante só acessa os próprios dados; o identificador vem do token, nunca do corpo da requisição |
| RN10 | Um lançamento retroativo altera o dia de competência e o saldo acumulado dos dias seguintes |

## 4.3 Requisitos não funcionais

Somente RNF01 e RNF02 vêm diretamente do desafio. As demais são metas propostas, e os valores medidos devem ser publicados no README após os testes.

| ID | Atributo | Meta | Como validar |
|---|---|---|---|
| RNF01 | Isolamento de falhas | Lançamentos continua operando com o Consolidado fora do ar | Derrubar o Consolidado durante o teste de carga |
| RNF02 | Vazão e perda | 50 req/s no Consolidado com até 5% de erro | k6 com taxa de chegada constante |
| RNF03 | Latência de consulta | p95 < 200 ms no pico | k6 |
| RNF04 | Latência de registro | p95 < 300 ms | k6 |
| RNF05 | Atualização do consolidado | p95 < 5 s em operação normal | Tempo entre `occurredAt` e a aplicação do evento |

| ID | Atributo | Meta | Como validar |
|---|---|---|---|
| RNF06 | Durabilidade | Nenhum lançamento confirmado é perdido | Reiniciar RabbitMQ e Consolidado e comparar totais |
| RNF07 | Consistência final | Consolidado igual à soma dos lançamentos após esvaziar a fila | Teste de reconciliação |
| RNF08 | Disponibilidade (alvo) | Lançamentos 99,9%; Consolidado 99,5% | SLOs e alertas |
| RNF09 | Recuperação (alvo) | RPO ≤ 5 min; RTO ≤ 30 min | Exercício de restauração |
| RNF10 | Escalabilidade | Serviços sem estado, escaláveis horizontalmente | Teste com mais de uma réplica |
| RNF11 | Segurança | JWT, TLS, isolamento por comerciante e auditoria | Testes de autorização e varredura de dependências |
| RNF12 | Observabilidade | Trace ponta a ponta, do HTTP ao consumidor | Verificação no Grafana |

## 4.4 Cenários de falha

| Falha | Comportamento esperado |
|---|---|
| Consolidado fora do ar | Lançamentos aceita registros; eventos aguardam na fila |
| RabbitMQ fora do ar | Lançamentos aceita registros; eventos aguardam no outbox |
| Redis fora do ar | Consultas vão ao Postgres, com maior latência |
| Banco de Lançamentos fora do ar | Registro retorna `503`; o cliente repete com a mesma chave |
| Evento duplicado | Ignorado pelo consumidor |
| Eventos fora de ordem | Mesmo resultado final, pois as somas são comutativas |
| Evento inválido | Vai para a DLQ, com alerta |

# 5. Arquitetura alvo

## 5.1 Contexto do sistema

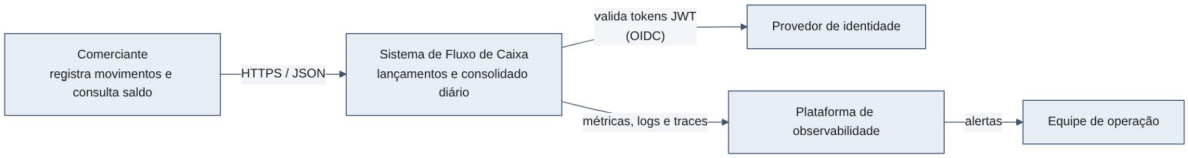



*Figura 1. Visão de contexto (C4, nível 1)*

## 5.2 Containers

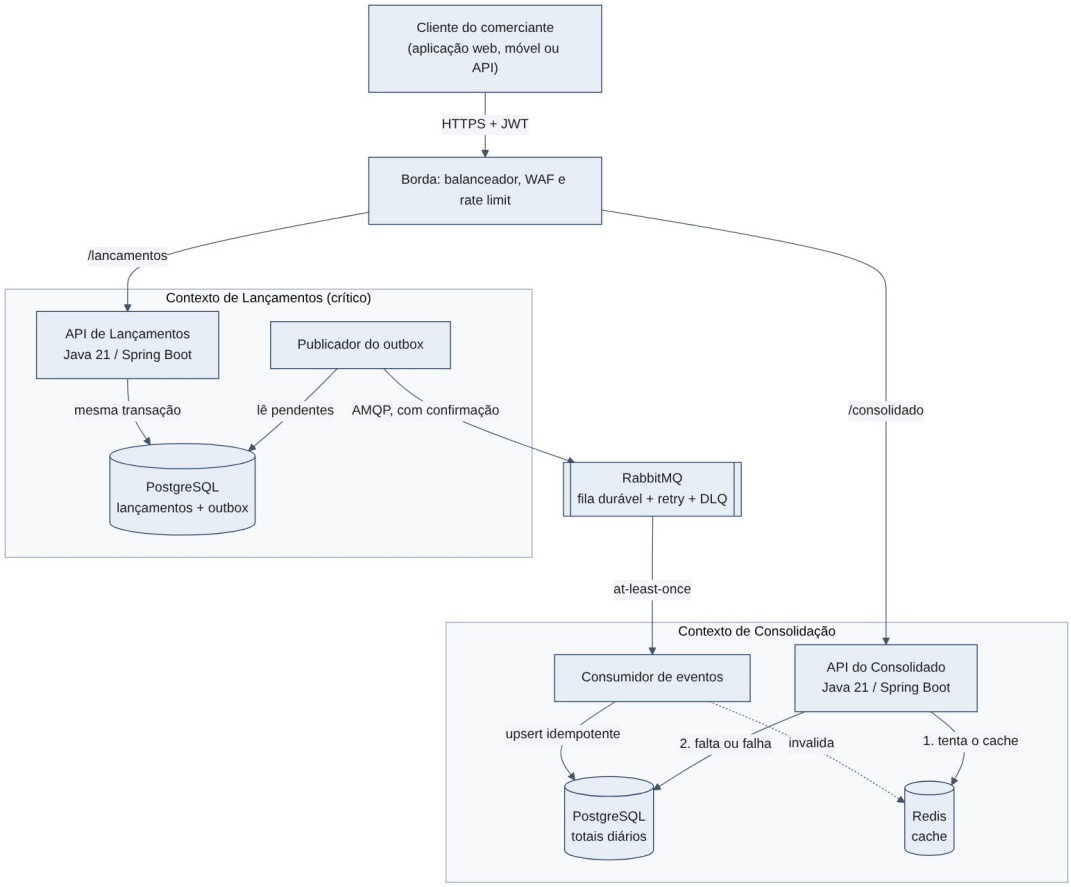



*Figura 2. Serviços, dados e integrações (C4, nível 2)*

| Componente | Responsabilidade |
|---|---|
| API de Lançamentos | Valida regras, garante idempotência e grava lançamento e evento na mesma transação |
| Publicador do outbox | Envia eventos pendentes ao broker e os marca como publicados após a confirmação |
| RabbitMQ | Entrega durável, retry e DLQ; absorve indisponibilidade do Consolidado |
| Consumidor de eventos | Aplica eventos de forma idempotente nos totais diários e invalida o cache |
| API do Consolidado | Serve saldos a partir do cache ou do banco próprio, sem chamar Lançamentos |
| Borda | TLS, WAF, rate limit e balanceamento |

## 5.3 Fluxo de registro

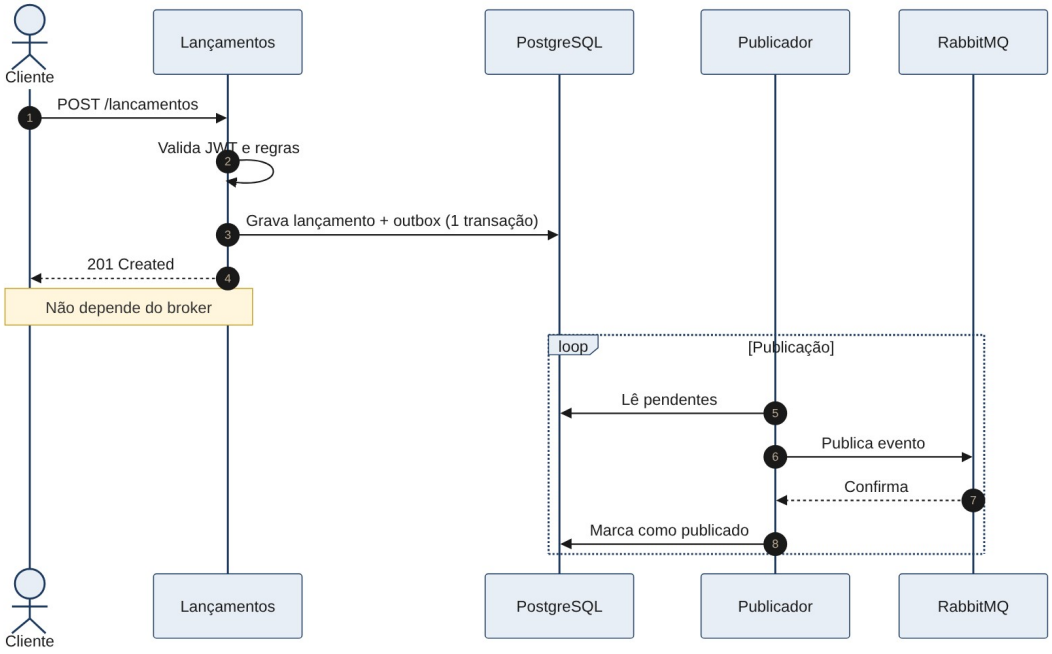



*Figura 3. Registro de lançamento com Transactional Outbox*

## 5.4 Fluxo de consolidação

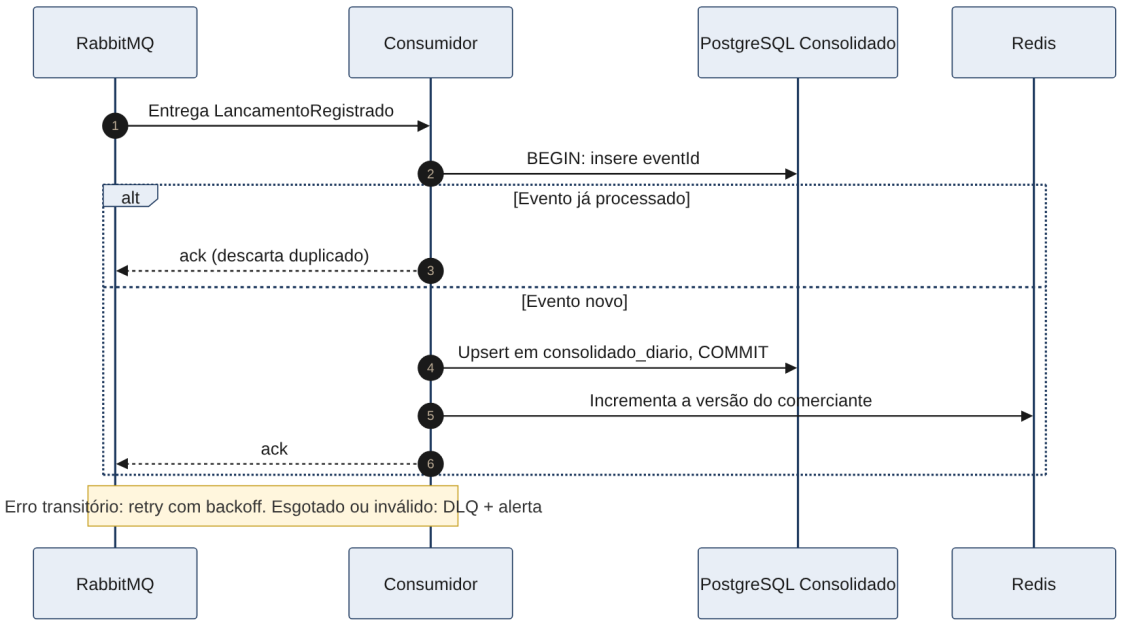



*Figura 4. Consumo idempotente e atualização do consolidado*

## 5.5 Consulta do saldo

1. A API valida o JWT e obtém o comerciante do token.
2. Procura a resposta no Redis, usando a versão atual do comerciante na chave.
3. Se não houver resposta em cache ou o Redis falhar, consulta `consolidado_diario` e soma os dias até a data pedida para obter o saldo acumulado.
4. Grava o resultado no cache, com TTL de 5 minutos, e responde.

## 5.6 Modelo de dados

| Tabela (serviço) | Colunas principais | Restrições |
|---|---|---|
| `lancamento` (Lançamentos) | id, comerciante_id, tipo, valor_centavos, data_competencia, estorno_de, criado_em | `valor_centavos > 0`; `estorno_de` único quando preenchido |
| `idempotencia` (Lançamentos) | comerciante_id, chave, hash_conteudo, lancamento_id | Única em (comerciante_id, chave) |
| `outbox` (Lançamentos) | event_id, tipo, payload, criado_em, publicado_em | Índice parcial nos pendentes |
| `consolidado_diario` (Consolidado) | comerciante_id, data, total_credito_centavos, total_debito_centavos | Chave primária (comerciante_id, data) |
| `eventos_processados` (Consolidado) | event_id, processado_em | Chave primária (event_id) |

## 5.7 APIs

| Endpoint | Descrição |
|---|---|
| `POST /lancamentos` | Registra um lançamento; exige `Idempotency-Key` |
| `POST /lancamentos/{id}/estorno` | Estorna um lançamento; exige `Idempotency-Key` |
| `GET /lancamentos/{id}` | Consulta um lançamento |
| `GET /lancamentos?de=&ate=&page=` | Lista lançamentos por período |
| `GET /consolidado/{data}` | Retorna créditos, débitos, saldo do dia e saldo acumulado |
| `GET /consolidado?de=&ate=` | Retorna o consolidado por período |

## 5.8 Estratégia de integração

| Trecho | Protocolo e formato | Motivo |
|---|---|---|
| Cliente e APIs | REST, HTTPS e JSON, com OpenAPI | Padrão simples e amplamente suportado |
| Lançamentos e Consolidado | AMQP e JSON, com `schemaVersion` | Desacopla disponibilidade e absorve picos |
| Serviços e identidade | OIDC e JWT | Validação local, sem chamada a cada requisição |

A evolução do contrato é compatível: campos novos são opcionais, e mudanças incompatíveis geram nova versão do evento.

## 5.9 Implantação na AWS

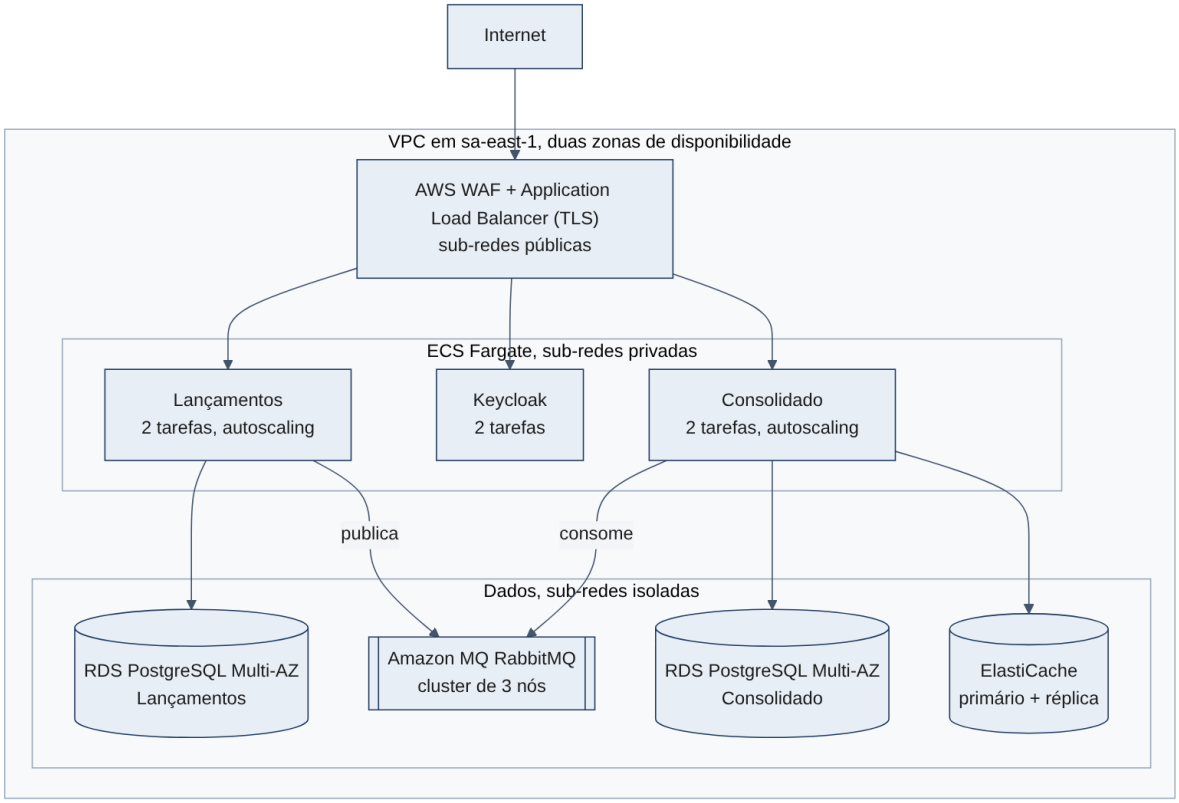



*Figura 5. Arquitetura alvo de implantação em duas zonas de disponibilidade*

O ambiente local usa os mesmos componentes em Docker Compose. A aplicação só depende de padrões abertos (AMQP, Postgres, Redis, OIDC e OpenTelemetry), o que reduz a dependência do provedor.

# 6. Decisões arquiteturais

Cada decisão também fica registrada como ADR em `docs/adr` no repositório.

## ADR-0001: Dois serviços independentes

**Contexto.** Lançamentos não pode cair com o Consolidado, e os perfis de carga são diferentes: um prioriza escrita durável; o outro, leitura em pico.

**Decisão.** Criar dois serviços implantáveis separadamente, cada um com seu banco, comunicando-se só por eventos.

| Alternativa | Por que não foi escolhida |
|---|---|
| Monolito | Mais simples, mas compartilha processo e recursos; um pico no relatório pode afetar o registro |
| Monolito modular com workers | Boa opção de baixo custo, com isolamento e escala independente mais fracos |
| Muitos microsserviços | Complexidade sem ganho: há apenas dois contextos de negócio |
| Serverless | Viável, mas dificulta a execução local e aumenta a dependência do provedor |

**Consequências.** Falhas e escala ficam isoladas, ao custo de dois deploys, um broker e consistência eventual.

## ADR-0002: Consolidação assíncrona com Transactional Outbox

**Contexto.** Uma chamada síncrona acoplaria a disponibilidade dos serviços. Gravar no banco e depois publicar cria escrita dupla: uma falha entre as etapas perde o evento.

**Decisão.** Gravar lançamento e evento na mesma transação. Um publicador lê o outbox com `FOR UPDATE SKIP LOCKED` e envia ao broker, marcando o evento só após a confirmação.

| Alternativa | Por que não foi escolhida |
|---|---|
| HTTP com circuit breaker | Evita a queda em cascata, mas não garante a entrega do evento |
| Publicação direta no broker | Sujeita à escrita dupla, e o registro passa a depender do broker |
| CDC com Debezium | Robusto, mas adiciona infraestrutura demais para este volume; é a evolução natural |
| Consulta ao banco de Lançamentos | Acopla os serviços ao esquema e à carga um do outro |

**Consequências.** Lançamentos segue operando com Consolidado ou broker fora do ar. A entrega é pelo menos uma vez, o consolidado tem consistência eventual monitorada, e o outbox precisa de rotina de limpeza.

## ADR-0003: RabbitMQ como broker

**Contexto.** É preciso entrega durável, retry e DLQ, com baixa complexidade e execução local simples. Não há, por ora, necessidade de replay longo nem de muitos consumidores.

**Decisão.** Usar RabbitMQ com exchange topic, quorum queue, publisher confirms, confirmação da mensagem só após o commit e DLQ. Na AWS, Amazon MQ for RabbitMQ.

| Alternativa | Por que não foi escolhida |
|---|---|
| Kafka | Excelente para alto volume e replay, mas superdimensionado neste caso |
| SQS/SNS | Muito simples de operar e mais barato, mas prende ao provedor e exige emulação local |
| Redis Streams | Reaproveita o Redis, mas mistura cache com dado que exige durabilidade |

**Consequências.** Operação e execução local ficam simples. Sem replay nativo, a reconstrução usa os lançamentos. Migrar é possível sem mudar o contrato do evento.

## ADR-0004: Consumidor idempotente e somas comutativas

**Contexto.** O broker pode entregar eventos duplicados ou fora de ordem, e duplicar um evento financeiro distorce o saldo.

**Decisão.** Na mesma transação, inserir o `eventId` em `eventos_processados` e fazer upsert somando o valor em `consolidado_diario`. Se o evento já existir, apenas confirmar a mensagem. O ack ocorre só após o commit. Erros transitórios têm retry com backoff; depois, a mensagem vai para a DLQ.

**Consequências.** O saldo converge mesmo com duplicidade e reordenação, e os consumidores escalam sem coordenar ordem. A tabela de eventos precisa de retenção maior que a janela máxima de reentrega.

## ADR-0005: Modelo de leitura com cache Redis

**Contexto.** Recalcular a partir de lançamentos a cada consulta seria custoso e dependeria de outro serviço. Além disso, um lançamento retroativo afeta o saldo acumulado dos dias seguintes.

**Decisão.** Persistir só totais diários e calcular o acumulado na consulta, para que um retroativo atualize uma única linha. Usar cache-aside com TTL de 5 minutos e chave `consolidado:{comerciante}:{versao}:{data}`, incrementando a versão ao aplicar um evento. Se o Redis falhar, consultar o Postgres.

| Alternativa | Por que não foi escolhida |
|---|---|
| Sem cache | Viável com índice adequado, mas com menos folga; o teste de carga deve medir a diferença |
| Saldo acumulado pré-calculado | Leitura direta, mas retroativos exigem atualização em cascata |
| Só TTL curto | Mais simples, mas pode servir saldo desatualizado e reduz o acerto de cache |

**Consequências.** A consulta não depende de Lançamentos nem do broker, e a falha do Redis só degrada a latência. Em dias de muitos lançamentos, a invalidação reduz o acerto de cache; se isso for medido, a evolução é agregar invalidações em janelas curtas.

## ADR-0006: Lançamentos imutáveis e idempotência na API

**Contexto.** Registros financeiros precisam de trilha de auditoria. Após um timeout, o cliente não sabe se houve gravação e tende a repetir a requisição.

**Decisão.** Não permitir edição nem exclusão; corrigir por estorno. Exigir `Idempotency-Key` com hash do conteúdo. Guardar valores em centavos, como `BIGINT`.

| Alternativa | Por que não foi escolhida |
|---|---|
| Edição com histórico | Mais complexa e exige propagar diferenças ao Consolidado |
| Exclusão lógica | Não representa bem o fato de negócio |
| Deduplicar por valor e horário | Pode rejeitar duas vendas legítimas de mesmo valor |
| Event sourcing completo | Poderoso, mas excessivo; a tabela imutável já oferece a auditoria necessária |

**Consequências.** O histórico fica completo e o consumidor só soma eventos. Os clientes precisam gerar e reutilizar a chave.

## ADR-0007: Java 21, Spring Boot e PostgreSQL

**Contexto.** O desafio permite qualquer linguagem. A escolha prioriza domínio técnico, maturidade do ecossistema e transações confiáveis.

**Decisão.** Java 21 (LTS) e Spring Boot, pela integração com AMQP, segurança e métricas; PostgreSQL, por ACID, upsert e `SKIP LOCKED`; Flyway para migrações; JUnit 5, Testcontainers e k6 para testes; OpenTelemetry, Prometheus e Grafana para observabilidade; Docker Compose para execução local.

| Alternativa | Por que não foi escolhida |
|---|---|
| Node.js ou Python | Viáveis, mas sem vantagem para este problema |
| Go | Consome menos recursos, sem necessidade comprovada para esta carga |
| Banco NoSQL | A transação entre lançamento e outbox é central, e o modelo relacional a atende diretamente |

**Consequências.** A JVM consome mais memória, o que influencia o custo. Virtual threads só devem ser adotadas se os testes mostrarem benefício.

## ADR-0008: OAuth2/JWT e isolamento por comerciante

**Contexto.** Dados de um comerciante não podem ser expostos a outro, e identidade é capacidade genérica, que não deve ser construída do zero.

**Decisão.** Delegar a autenticação a um provedor OIDC, com Keycloak. Validar o JWT em cada serviço, usar escopos por operação e obter o `comercianteId` apenas do token.

| Alternativa | Por que não foi escolhida |
|---|---|
| API key | Simples, mas sem expiração curta, escopos ou identidade padronizada |
| Autenticação própria | Maior risco e nenhum diferencial para o negócio |
| Validar só no gateway | Sem defesa em profundidade: um acesso interno contornaria o controle |

**Consequências.** O isolamento fica claro e testável, com um componente a mais no ambiente. mTLS fica como evolução, pois os serviços não fazem chamadas síncronas entre si. Para simplificar a execução local, os tokens vêm de um emissor de desenvolvimento com chave HS256, ativo só no perfil local; o provedor OIDC é ativado por configuração.

# 7. Atributos de qualidade

## 7.1 Escalabilidade e desempenho

- **Horizontal:** as APIs não guardam estado; o balanceador distribui as requisições entre as réplicas.
- **Dimensionamento N+1:** o Consolidado parte de 2 tarefas, e cada uma deve suportar sozinha os 50 req/s, para que a perda de uma réplica não viole o requisito. O teste de carga precisa confirmar esse dimensionamento.
- **Autoscaling:** proposta inicial de 2 a 6 tarefas, com alvo de 60% de CPU, a calibrar com os testes.
- **Cache:** reduz a carga do banco em leituras repetidas.
- **Consumo independente:** o consumidor pode escalar conforme a profundidade da fila, e a fila absorve picos de escrita.
- **Banco:** índice pela chave `(comerciante_id, data)`; réplicas de leitura ficam como evolução, se as medições indicarem necessidade.

## 7.2 Resiliência

| Mecanismo | Aplicação |
|---|---|
| Redundância | Tarefas, banco, broker e cache distribuídos em duas zonas de disponibilidade |
| Failover | RDS Multi-AZ, quorum queues e réplica do ElastiCache |
| Isolamento | Banco, pool de conexões e implantação separados por serviço |
| Timeouts e backoff | Em todas as dependências; retry somente em operações idempotentes |
| Degradação controlada | Sem Redis, leitura no Postgres; sem broker, acúmulo no outbox |
| Recuperação de dados | Backups automáticos com restauração a um ponto no tempo e reconstrução do consolidado a partir dos lançamentos |
| Reprocessamento | DLQ com alerta e procedimento de reenvio após a correção |

## 7.3 Segurança e critérios de consumo

| Critério | Definição |
|---|---|
| Autenticação | OAuth2 e OIDC; `client credentials` para sistemas e `authorization code` com PKCE para usuários |
| Tokens | JWT de curta duração; assinatura, emissor, audiência e validade verificados em cada serviço |
| Autorização | Escopos `lancamentos:write`, `lancamentos:read` e `consolidado:read` |
| Isolamento | `comercianteId` vem do token e filtra todas as consultas, prevenindo acesso indevido a objetos de outro comerciante |
| Criptografia | TLS 1.2 ou superior em trânsito e KMS em repouso |
| Proteção da borda | WAF e rate limit por cliente, com limite global acima do pico de 50 req/s |
| Validação | Esquema OpenAPI, limites de tamanho e rejeição de campos inválidos |
| Rede e segredos | Dados em sub-redes isoladas, privilégio mínimo por serviço e segredos fora do código |
| Auditoria | Lançamentos imutáveis e logs com usuário, operação e `traceId`, sem tokens ou dados sensíveis |
| Cadeia de suprimentos | Varredura de dependências e imagens no pipeline |
| Privacidade (LGPD) | Coleta mínima de dados pessoais, retenção definida e acesso restrito |

# 8. Observabilidade

A instrumentação usa OpenTelemetry. O contexto de trace viaja nos cabeçalhos da mensagem, ligando a requisição HTTP ao processamento do evento.

| Sinal | O que medir | Alerta proposto |
|---|---|---|
| Consultas | Taxa, erros e latência p95 do Consolidado | Erro acima de 5% por 5 min; p95 acima de 200 ms por 10 min |
| Registros | Taxa, erros e latência p95 de Lançamentos | Erro acima de 1% por 5 min |
| Outbox | Eventos pendentes e idade do mais antigo | Mais antigo acima de 1 min |
| Fila | Profundidade e idade das mensagens | Crescimento contínuo por 10 min |
| DLQ | Quantidade de mensagens | Qualquer mensagem |
| Atraso do consolidado | Tempo entre `occurredAt` e a aplicação | p95 acima de 5 s por 10 min |
| Cache | Taxa de acerto e falhas do Redis | Falhas contínuas por 5 min |
| Reconciliação | Diferença entre lançamentos e consolidado | Qualquer diferença após esvaziar a fila |
| Infraestrutura | CPU, memória, conexões e armazenamento | Saturação acima de 80% por 15 min |

Os limites acima são pontos de partida, a calibrar após medir o sistema. Os alertas de maior severidade devem apontar para um procedimento de atuação (runbook), e os logs estruturados em JSON devem conter `traceId`, `comercianteId` e `eventId`.

# 9. Estratégia de testes

| Nível | O que valida | Ferramenta |
|---|---|---|
| Unitário | Regras de negócio, estorno e cálculo de saldo | JUnit 5 |
| Integração | Outbox, consumidor, upsert e cache com dependências reais | Testcontainers |
| Contrato | Compatibilidade dos eventos e da API | Validação de esquema |
| Segurança | Acesso entre comerciantes, escopos e tokens inválidos | Testes de API |
| Carga | RNF02, RNF03 e RNF04 | k6 |
| Resiliência | RNF01, RNF06 e RNF07 | k6 e Docker Compose |

## Provas dos requisitos do desafio

Os resultados serão preenchidos após a execução, sempre informando o ambiente utilizado. Um teste local demonstra o comportamento, mas não substitui a validação no ambiente alvo.

| Cenário | Critério de aprovação | Resultado |
|---|---|---|
| 50 req/s de consultas por 10 minutos | Erros ≤ 5%; p95 < 200 ms | A medir |
| Mesma carga com uma réplica derrubada | Erros ≤ 5% | A medir |
| Mesma carga com o Redis fora do ar | Erros ≤ 5% | A medir |
| Consolidado fora do ar durante registros | 100% dos registros válidos aceitos | A medir |
| RabbitMQ reiniciado durante registros | Nenhum lançamento confirmado perdido | A medir |
| Reconciliação após recuperação | Consolidado igual à soma dos lançamentos | A medir |
| Eventos duplicados e fora de ordem | Saldo correto | A medir |

# 10. Arquitetura de transição

O desafio não descreve um legado, portanto esta seção assume um sistema existente que concentra lançamentos e relatórios em uma única base. A estratégia é Strangler Fig: migrar por comerciante, de forma gradual e reversível.

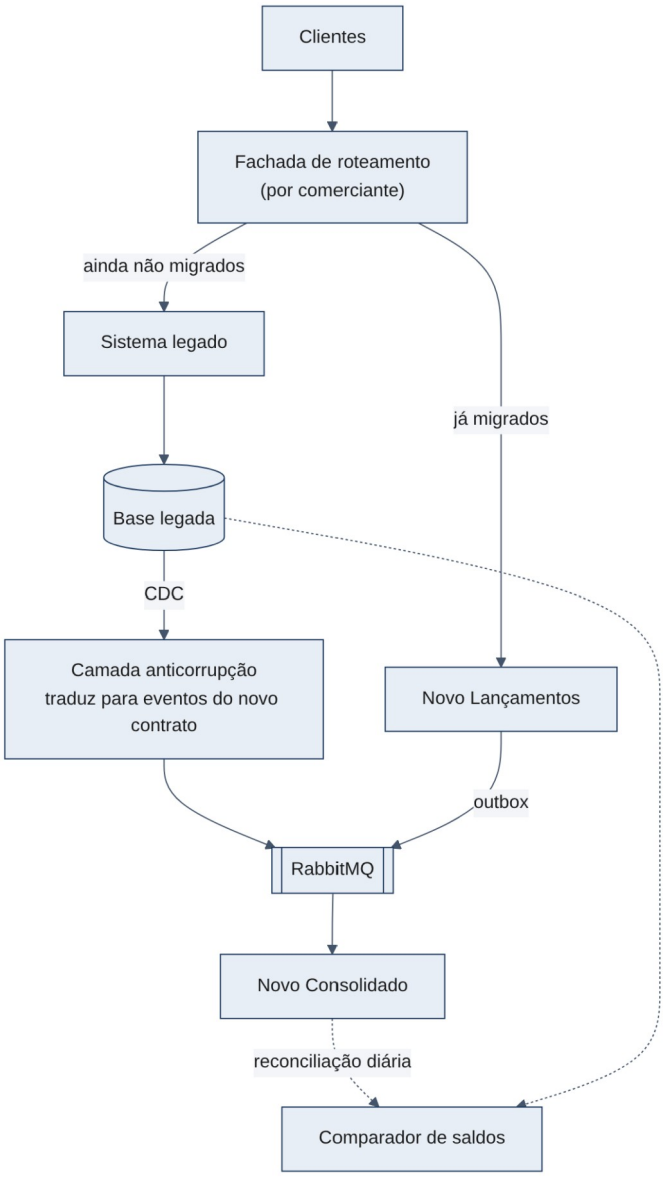



*Figura 6. Convivência entre legado e nova arquitetura*

## Fases da migração

| Fase | Atividades | Critério para avançar |
|---|---|---|
| 0. Preparação | Mapear regras e dados, publicar a fachada sem alterar o destino | Tráfego passando pela fachada sem impacto |
| 1. Carga e sombra | Fazer a carga histórica e alimentar o novo Consolidado por CDC | Reconciliação sem diferenças por período acordado |
| 2. Consulta | Mover leituras de um grupo piloto para o novo Consolidado | Metas de latência e erro atendidas |
| 3. Escrita | Migrar a escrita por comerciante para o novo Lançamentos | Piloto estável e plano de retorno testado |
| 4. Desligamento | Concluir a migração, arquivar o legado e remover CDC e fachada | Todos os comerciantes migrados e dados arquivados |

## Cuidados

- **Um único escritor:** cada comerciante escreve em apenas um sistema por vez, evitando escrita dupla.
- **Retorno:** até o desligamento, os lançamentos novos também são replicados ao legado, para permitir o retorno.
- **Camada anticorrupção:** o modelo do legado não contamina o novo domínio.
- **Idempotência:** o `eventId` do CDC é derivado da chave do legado, evitando duplicidade em reprocessamentos.

# 11. Estimativa de custos

A estimativa é uma **ordem de grandeza de US$ 1.450 por mês**, com faixa de US$ 1.020 a US$ 1.890, para produção em São Paulo (sa-east-1), com preços sob demanda e sem descontos. As licenças não têm custo, pois todos os componentes escolhidos têm versão gratuita ou de código aberto. Suporte da AWS, ambientes de não produção, impostos e câmbio não estão incluídos.

| Item | Dimensionamento | Mês | Base do cálculo |
|---|---|---|---|
| Computação: Lançamentos e Consolidado | 4 tarefas Fargate de 1 vCPU e 2 GB | US$ 216 | US$ 0,0494/h por tarefa, sem região indicada, × 1,5 |
| Identidade: Keycloak | 2 tarefas Fargate de 1 vCPU e 2 GB | US$ 108 | Mesma base da computação |
| Banco de dados | 2 RDS PostgreSQL db.t4g.medium Multi-AZ | US$ 400 | US$ 100/mês em sa-east-1 por instância, × 2 para Multi-AZ |
| Armazenamento dos bancos | 50 GB por banco, em 2 zonas (200 GB) | US$ 30 | US$ 0,152/GB-mês, preço de EBS gp3 em sa-east-1 usado como aproximação |
| Mensageria | Amazon MQ RabbitMQ, 3 nós mq.m7g.medium | US$ 392 | US$ 261/mês em us-east-1 × 1,5 |
| Cache | ElastiCache cache.t4g.small, 2 nós | US$ 56 | US$ 0,0256/h em us-east-1 × 1,5 |
| Rede e borda | ALB, NAT, WAF e tráfego de saída | US$ 150 | Reserva estimada, não cotada |
| Observabilidade e segredos | Logs, métricas, traces, Secrets Manager e KMS | US$ 100 | Reserva estimada, não cotada |
| **Total** | | **US$ 1.453** | 730 horas por mês |

**Como ler o número.** Nem todos os preços de São Paulo foram encontrados. O fator de 1,5 é uma premissa baseada na diferença citada para o Amazon MQ, e duas linhas são reservas. Antes de qualquer decisão, os valores devem ser recalculados na AWS Pricing Calculator, com o dimensionamento obtido nos testes de carga.

## Alavancas de redução

- Usar Savings Plans e instâncias reservadas, caso a carga se mostre estável.
- Usar processadores ARM na computação, após validar o desempenho.
- Trocar o Amazon MQ por SQS/SNS, reduzindo uma das maiores linhas, ao custo da portabilidade (ADR-0003).
- Usar um provedor de identidade gerenciado, se o custo for menor que operar o Keycloak.

**Fontes de preços**, consultadas em 5 de novembro de 2026 e, em sua maioria, agregadores, não a tabela oficial da AWS: cloudpricecheck-com.pages.dev/aws/fargate-pricing (Fargate); bytebase.com/dbcost/rds/instance/db.t4g.medium.md (RDS); aws-pricing.com/sa-east-1.html (EBS); cloudburn.io/blog/amazon-mq-pricing (Amazon MQ); cloudprice.net/aws/elasticache?region=us-east-1 (ElastiCache).

# 12. Repositório e execução local

**Repositório:** https://github.com/danielbrazil28/verx.git

## Estrutura

```
fluxo-caixa/
├── README.md
├── docker-compose.yml
├── docs/
│   ├── 01-dominios-e-capacidades.md
│   ├── 02-requisitos.md
│   └── adr/
├── lancamentos-service/
├── consolidado-service/
├── load-tests/
└── observability/
```

## Execução local

O README traz as instruções completas. A execução usa Docker; os testes exigem também Java 21 e Maven.

```
docker compose up -d --build
mvn verify
k6 run load-tests/consolidado-pico.js
```

# 13. Riscos e evoluções futuras

## Riscos

| Risco | Impacto | Mitigação |
|---|---|---|
| Saldo temporariamente desatualizado | Médio | Monitorar o atraso e mostrar a data da última atualização |
| Crescimento do outbox com broker fora do ar | Médio | Alerta por idade e limpeza dos publicados |
| Evento preso na DLQ deixar o saldo incorreto | Alto | Alerta imediato, reprocessamento e reconciliação |
| Invalidação frequente reduzir o acerto de cache | Baixo | Medir e agregar invalidações, se necessário |
| Soma do acumulado ficar lenta com histórico longo | Baixo | Criar fotografias mensais de saldo |
| Custo do broker gerenciado em volume baixo | Médio | Reavaliar SQS/SNS |

## Evoluções

- Reconstrução automatizada do consolidado por comerciante (RF07).
- Substituir o polling do outbox por CDC, se a latência ou o volume justificarem.
- Kafka, caso surjam novos consumidores e necessidade de replay.
- Novas capacidades, como previsão de fluxo de caixa, conciliação bancária e categorização de lançamentos.
- Infraestrutura como código, pipeline com implantação canário e testes de caos.